Enhanced FIWARE-Based Architecture for Cyberphysical Systems With Tiny Machine Learning and Machine Learning Operations: A Case Study on Urban Mobility Systems

Javier Conde, Andrés Munoz-Arcentales, Álvaro Alonso, Joaquín Salvachúa, and Gabriel Huecas,
Universidad Politécnica de Madrid, 28040, Madrid, Spain

The rise of AI and the Internet of Things is accelerating the digital transformation of society. Mobility computing presents specific barriers due to its real-time requirements, decentralization, and connectivity through wireless networks. New research on edge computing and tiny machine learning (tinyML) explores the execution of AI models on low-performance devices to address these issues. However, there are not many studies proposing agnostic architectures that manage the entire lifecycle of intelligent cyberphysical systems. This article extends a previous architecture based on FIWARE software components to implement the machine learning operations flow, enabling the management of the entire tinyML lifecycle in cyberphysical systems. We also provide a use case to showcase how to implement the FIWARE architecture through a complete example of a smart traffic system. We conclude that the FIWARE ecosystem constitutes a real reference option for developing tinyML and edge computing in cyberphysical systems.

The growth of the Internet of Things (IoT) has enabled the deployment of a multitude of devices throughout the world. By 2021, there were 12.2 billion active devices, and this number is expected to increase to around 27 billion connected devices by 2025.[1] The IoT, in combination with other technologies, such as cloud computing, big data, and AI, has enabled the expansion of digital twins (DTs) and cyberphysical systems (CPSs).[2] Both DTs and CPSs involve the integration of the digital and physical world through a bidirectional communication: 1) The physical entity collects information from the real world. 2) It transmits this information to the virtual environment. 3) The virtual entity processes the data. 4) The virtual entity sends commands to the physical entity. 5) The physical entity updates its state. CPSs have been applied to all societal activities, including complex systems with specific requirements, such as cyberphysical mobile systems (e.g., smart vehicles) located in dynamic environments (e.g., smart cities).[3]

These systems have a set of requirements that need to be addressed, such as real-time processing, guaranteeing reliability and security, handling large amounts of data, interdevice communication, and data ingestion standardization from heterogeneous environments.[4] In the literature, there are proposals aimed at providing solutions to these challenges. Conde et al.[5] propose the FIWARE ecosystem as an open source solution for the implementation of scalable, AI-driven, and use-case-agnostic DTs and CPSs. The FIWARE initiative[a] is defined as an ecosystem of software components that facilitate the development of intelligent solutions. Every FIWARE solution is based on communication using the Next Generation Service Interface version 2 (NGSIv2[b]) or NGSI for linked data (NGSI-LD[c]) data formats. A central component called context

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/
Digital Object Identifier 10.1109/MITP.2024.3421968
Date of current version 1 October 2024.

[a]FIWARE: https://www.fiware.org/
[b]NGSIv2: https://swagger.lab.fiware.org/
[c]NGSI-LD: https://forge.etsi.org/rep/NGSI-LD/NGSI-LD

TABLE 1. Role of FIWARE components in the development of CPSs.

Component	Function
NGSI-LD	Data format homogenization
Smart data models	Data models homogenization
Orion	Real-time communication
IoT agents	Communication with IoT devices
Draco	Data ingestion
Keyrock and Wilma	Authentication and authorization
Cosmos	Big data and machine learning processing

broker[d] acts as the centerpiece of the solution in charge of managing the context information. It implements the NGSI application programming interface and offers synchronous and asynchronous communication capabilities. The other components communicate with the context broker, providing additional functionalities to the CPS.

Table 1 summarizes some FIWARE components and their roles in the CPS. The reference architecture for DTs and CPSs[5] has been validated in real scenarios, such as the Airport of Aberdeen.[6] However, the proposal of Conde et al.[5] processes all of the information in the cloud, including the execution of AI models. Consequently, it is necessary to keep IoT devices connected to the rest of the infrastructure, causing high latencies, high server demands, and service loss in the case of a communication failure.

Other investigations study architectures in which data are processed in a location close to the data sources. This paradigm, known as edge computing, has advantages, such as reduced latency, energy savings, scalability, and security by not having to send data over the network.[7] Particular cases of edge computing include EdgeAI and tiny machine learning (tinyML), which involve the deployment of AI models on the edge or directly on low-performance devices.[8]

TinyML investigation has opened new research lines in hardware optimization, software, and AI, such as model compression, model quantization, or model pruning. Thanks to tinyML, it is now possible to execute complex models on tiny devices.[9,10]

Although tinyML is progressing very fast, it does not mean that IoT devices must be isolated. IoT devices may receive information from the cloud that they are not capable of obtaining or processing. Additionally, the IoT devices may communicate with other systems. There needs to be a balance between processing on the device and processing in the cloud. To achieve this, the tinyML deployment group proposes ML operations (MLOps) as an approach to managing the tinyML lifecycle.[11]

The article is structured as follows. In the next section, we extend the architecture based on the FIWARE initiative for DTs and CPSs[5] to integrate tinyML and MLOps into it and explain its benefits. In the third section, we apply our proposal to an easy-to-follow use case of a smart traffic barrier. Lastly, conclusions and future work arising from our research are presented.

EXTENDING THE FIWARE ARCHITECTURE TO INTEGRATE THE MLOps LIFECYCLE AND tinyML IN CPSs

Obtaining efficient ML models is not the only task in tinyML. Kolltveit and Li[12] differentiate between two phases: development and operationalization. In the development phase, the ML model is trained and evaluated. It requires batch processing of large amounts of data. In the operationalization phase, the model is loaded onto the final device that is the one that executes it. The MLOps methodology establishes a fine-grained division of all tasks involving the ML lifecycle. According to Barrak et al.,[13] the MLOps process includes 1) data retrieval, 2) data preparation, 3) model training, 4) evaluation, 5) tuning, 6) deployment, and 7) monitoring. These phases coincide with the requirements defined by the tinyML Foundation[11] regarding the tinyML lifecycle.

Figure 1 shows the MLOps cycle adapted to the tinyML scenario, including the transformation from the traditional models to their lite versions, capable of being run on low-performance devices without excessive loss of the prediction quality.

In the next sections, we present the results of our investigation showing how the FIWARE architecture for DTs and CPSs can be extended to implement the MLOps cycle and adapt it to the tinyML scenario. We explain the benefits of our proposal and how it overcomes the barriers mentioned in the literature.[4]

Collecting Data From IoT Devices

The first phase consists of generating historical data to train ML models. The historical data come from measurements of the devices themselves, so it is necessary to define an architecture for data collection, data homogenization, and data storage. According to

[d]Orion CB: https://github.com/FIWARE/context.Orion-LD

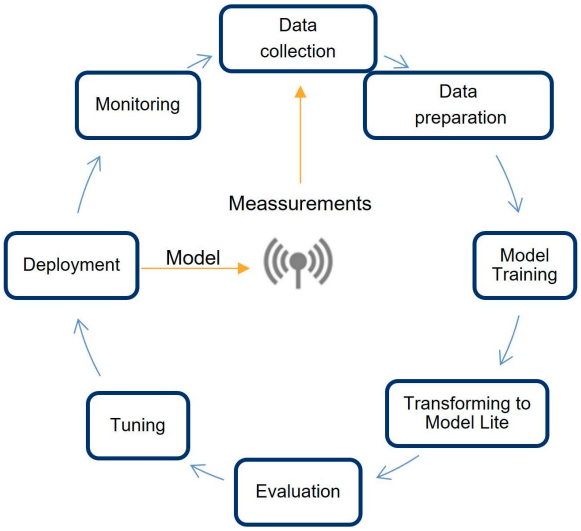


FIGURE 1. MLOps cycle adapted to tinyML.

Schizas et al.,[9] data collection from sensors is a required task when there are not specific datasets to the use case or there are not datasets with the level of quality needed.

FIWARE offers a set of components called IoT agents (IDAS) that facilitate the integration of external systems, including IoT devices, into the FIWARE ecosystem. IoT agents act as middleware between the context broker and the physical device. That is, they translate NGSI-LD HTTP requests coming from the context broker into the native data format and protocol of the IoT device. The context broker communicates with the IoT device through commands processed by the IoT agent, and the IoT agent communicates with the context broker by updating the properties of the corresponding NGSI-LD entity.

IoT agents ease the dataset generation but also the deployment phase, which is transparent to the IoT devices. The information will be managed by the Orion-LD context broker. This proposal also solves the problem of integrating heterogeneous sources in terms of communication protocols, simplifying the rest of the MLOps phases, as all of the data will be accessible through the NGSI-LD format.

Modeling and Storing Data

Data in the CPS are modeled through NGSI-LD, which is agnostic to the use case. The FIWARE Smart Data Models Initiative[e] defines specific data models for diverse smart domains. In this sense, NGSI-LD allows the homogenizing of data formats, while the FIWARE smart data models allow the homogenizing of data models. Both are essential for complex systems, such as a CPS, where many actors are involved.

FIWARE Draco,[f] based on Apache Nifi, is a high-scalable tool for transforming and routing data in real time. It is used to integrate systems different from IoT devices in the CPS and to generate historical data for the training phase.

Training Data and Managing ML Versions

The training phase can begin once the historical data are available. Different frameworks and libraries adapted to tinyML scenarios are mentioned in the literature (e.g., TensorFlow Lite, uTensor, and Pythorch Mobile). One of the most widely used is TensorFlow Lite[g] because of its efficiency in compressing models and adapting them to a wide variety of microcontrollers. TensorFlow Lite is focused on neural networks. Emlearn[h] is a library that converts the model trained through the Keras[i] or Scikit-learn[j] libraries to C code. Emlearn is compatible with any device that includes a C99 compiler. It has been tested on microcontrollers, such as the ESP32, ESP8266, and STM32.

ML does not follow a linear cycle; it is an iterative process. Better models can be obtained by tuning the hyperparameters, changing the algorithm, increasing the quality and/or quantity of historical data, etc. We have identified the necessity of ML orchestrators that help to monitor and automate all phases of the MLOps lifecycle. Open source tools, such as MLFlow[k] or Neptune,[l] allow us to manage the different phases of MLOps. These types of tools ease the deployment, tracking, and monitoring of ML models in IoT devices as well as support reuse, collaboration, maintenance, and replicability of the scenario.

A CPS is a complex system, with many components participating in it. Consequently, we have also identified the necessity of high-level orchestrators that control the entire scenario. In contrast to ML orchestrators that only manage the MLOps cycle, high-level orchestrators can manage every process of the CPS. Tools such as Apache Airflow,[m] Perfect,[n] or Flyte[o] are candidates for high-level orchestrators.

[e]Smart Data Models: https://smartdatamodels.org/

[f]FIWARE Draco: https://github.com/ging/fiware-draco
[g]TensorFlow: https://www.tensorflow.org
[h]Emlearn library: https://github.com/emlearn/emlearn
[i]Keras Library: https://keras.io/
[j]Scikit-learn library: http://scikit-learn.org/
[k]MLFlow: https://mlflow.org/
[l]Neptune: https://mlops.neptune.ai/
[m]Apache Airflow: https://airflow.apache.org/
[n]Perfect: https://www.prefect.io/
[o]Flyte: https://flyte.org/

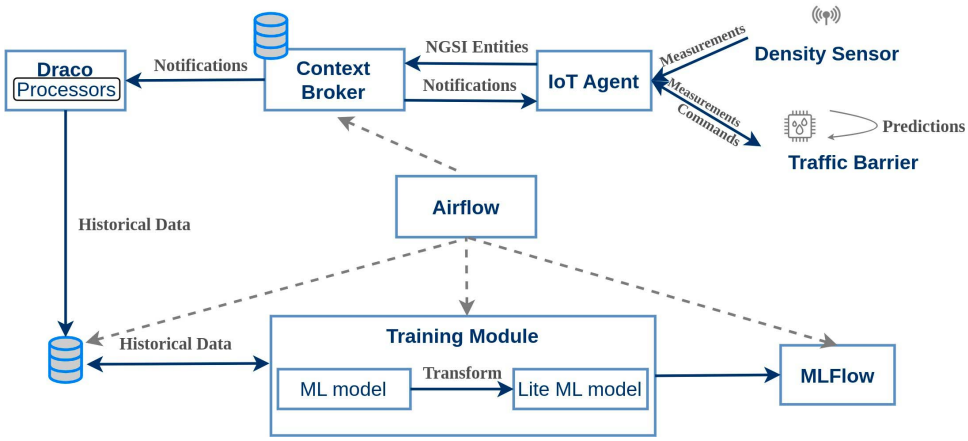


FIGURE 2. General architecture of the smart traffic system.

As part of our research, we have integrated Apache Airflow and MLFlow in the original FIWARE architecture.[p]

Our proposal enhances the state of the art, where FIWARE-based architectures for CPS with AI capabilities delegate the prediction tasks to the core of the network, resulting in additional latency, network overhead, and the need to connect IoT devices to the cloud.[5,14] Furthermore, these proposals do not integrate the ML lifecycle within the architecture, making them less reproducible and less extensible to new model versions. Similarly, other works do explore the integration of tinyML and MLOps,[9,15] but they do not include the phase of information retrieval and IoT management, which is essential to complete the MLOps cycle in a CPS. Our proposal, thus, addresses all of these barriers in a single architecture that enables the improvement of CPS operation in an automatable, secure, and real-time manner.

Deployment and Monitoring

Once an efficient and device-compatible model has been obtained, it is necessary to define a mechanism to deploy it. The IoT agents can send commands to be executed on the IoT devices and notify them of the existence of a new model. This approach makes it easy to keep the sensor up to date and facilitates the automation of the process.

Regarding monitoring, the device can also send its predictions to the context broker through its respective IoT agent. With these results, the quality of the trained model can be evaluated, and the MLOps lifecycle can be fed back by generating new versions of the model.

[p]MLOps supermarket use case: https://github.com/ging/fiware-mlops-supermarket

USE CASE OF A tinyML–MLOps PIPELINE FOR A SMART TRAFFIC SYSTEM

Given that FIWARE technology is agnostic to the use case and that the Smart Data Models Initiative covers a wide range of scenarios,[14] this makes our proposal implementable in any field, including smart agriculture, smart aerospace, smart energy, etc.

As an example of implementation, we present a use case for an automatic traffic barrier system that serves as a reference guide for the integration of the MLOps process for tinyML in CPS mobility systems. This scenario contains two sensors that are not connected to each other. The first detects the density of vehicles in a specific area. The second is a smart traffic barrier that controls the traffic of such an area.

The main goal is to train a machine learning model and deploy it in the traffic control system. The model predicts the density of vehicles in the area, and, based on the result obtained, the smart traffic barrier allows vehicles to pass or not. The scenario is dockerized so it can be easily replicated and deployed. All of the code is available in a public repository.[q]

Figure 2 shows the FIWARE component-based architecture for the TinyML–MLOps lifecycle adapted to the traffic system use case.

Data Collection and Generation of the Dataset

In the pretraining phase, it is necessary to generate historical data if they did not exist previously. In this scenario, data were collected from a loop situated on a

[q]FIWARE TinyML and MLOps traffic use case: https://github.com/ging/tinyML-MLOps-Fiware-Barrier

road of Santander, Spain, which records the density of vehicles (vehicles/minute) passing over it. The dataset also includes meteorological information from Santander provided by the Spanish government. Data were collected throughout half of the year of 2021, with one measurement every 15 minutes. Draco was used to generate the historical data. Upon changes in real-time NGSI-LD entities, Draco receives notifications, processes them, and stores the results in a Mongo database, which was used to generate the training comma-separated values file.

Training the Density of Vehicles Model

Once the information is stored in Mongo, the data are cleaned and processed. The predictor uses the environment data, date, and time information to train a model that classifies the density as low when there are fewer than 10 vehicles per minute and as high in the opposite case. The binary classifier is a random forest (RF) implemented by the scikit-learn library. First, we trained a model named large_model, configuring the RF with 50 trees and a maximum depth of 10, resulting in a model accuracy of 91.4%. This model took 301.83 ms to train and 9.18 ms to make 3267 predictions (with an average of 2.81 μs per prediction), and it occupies 2.79 MB. The resultant model is transformed by the emlearn library so that it can be deployed and executed on the barrier controller.

The tiny model was compressed by reducing the RF to 10 trees, with a maximum depth of eight levels, achieving an accuracy of 89.7% (1.7% lower than the uncompressed model). It took 65.54 ms to train and transform the model (78.3% less time) and 1.61 ms to make 3267 predictions (with an average of 0.49 μs per prediction), and it occupies 0.13 MB (95.35% smaller than the large_model.), making the model fit within an ESP32 with 4 MB of memory. The training process is managed within MLFlow. MLFlow generates metadata of the process, such as the algorithm, hyperparameters, link to the historical data, evaluation results, or version. The outcome of this module is a new version of the ML model ready to be deployed on the smart traffic barrier.

Execution of the Model in the Smart Traffic Barrier

Once the data model has been generated, it can be uploaded to the smart traffic barrier, and it can start operating. In a traditional approach, the prediction would be made in the cloud, and the result would be transmitted to the smart traffic barrier. In contrast, with our proposal based on tinyML, the model is loaded onto the IoT device, which executes the prediction internally, saving battery and time, without interacting with other systems.

The whole process is automated through Airflow. A directed acyclic graph schedules a new ML cycle by training and deploying a new tinyML model when new data are collected. In our scenario, the model was retrained with a dataset covering all of 2021 instead of just half the year. The newly compressed model improved its accuracy to 93.1%, took 103 ms to train, and occupies 0.13 MB.

CONCLUSION AND FUTURE WORK

The evolution of the IoT has motivated the trend of bringing computing closer to the edge of the network, including the execution of ML models. Most tinyML research focuses on hardware improvement and the generation of efficient models to transfer processing to the IoT device. However, a complete architecture that performs all prerequisites through MLOps must be implemented. In this article, we propose extending the FIWARE ecosystem, which has already been validated to implement DTs and CPSs, by incorporating tinyML and MLOps capabilities. We explained the benefits of our proposal, which fulfills the requirements for a CPS deployed in heterogeneous environments. Moreover, this article has a practical approach, including the use case of a smart traffic system. From our work, we can conclude that we have enhanced the FIWARE capabilities for developing CPSs by integrating tinyML capabilities managed by the MLOps lifecycle.

For future research, the architecture can be extended with new MLOps and high-level orchestrators, and it will be validated within new use cases from different sectors. Furthermore, the architecture presented can be distributed through a federated context broker with the aim of bringing the training phase as close as possible to the IoT device. Another future line is related to the integration of large language models (LLMs) on edge devices. New research should focus on optimizing LLMs, ensuring an automatic, fast, and accurate deployment in CPSs while minimizing resource demands.

ACKNOWLEDGMENTS

This work was supported by the Future Networks for Datacenters and Telcos (FUN4DATE) (PID2022-136684OB-C22) project funded by the Spanish Agencia Estatal de Investigación and by the Chips Act Joint Undertaking project Scalable and Quantum Resilient Heterogeneous Edge Computing enabling Trustworthy AI (SMARTY) (Grant 101140087).

REFERENCES

1. M. Hasan. "State of IoT 2022: Number of connected IoT devices growing 18% to 14.4 billion globally." IoT Analytics. Accessed: Jan. 28, 2024. [Online]. Available: https://iot-analytics.com/number-connected-iot-devices
2. Z. Song, A. R. Mishra, and S. P. Saeidi, "Technological capabilities in the era of the digital economy for integration into cyber-physical systems and the IoT using decision-making approach," *J. Innov. Knowl.*, vol. 8, no. 2, 2023, Art. no. 100356, doi: 10.1016/j.jik.2023.100356.
3. S. Kaliappan and R. Maranan, "Implementation of cyber physical systems in smart cities through augmented reality networks in the mobility decade," in *Cyber-Physical Systems and Supporting Technologies for Industrial Automation*. Hershey, PA, USA: IGI Global, 2023, pp. 215–228.
4. K. T. Chui et al., "A survey of internet of things and cyber-physical systems: Standards, algorithms, applications, security, challenges, and future directions," *Information*, vol. 14, no. 7, 2023, Art. no. 388, doi: 10.3390/info14070388.
5. J. Conde, A. Munoz-Arcentales, A. Alonso, G. Huecas, and J. Salvachúa, "Collaboration of digital twins through linked open data: Architecture with FIWARE as enabling technology," *IT Prof.*, vol. 24, no. 6, pp. 41–46, 2022, doi: 10.1109/MITP.2022.3224826.
6. J. Conde et al., "Applying digital twins for the management of information in turnaround event operations in commercial airports," *Adv. Eng. Inform.*, vol. 54, Oct. 2022, Art. no. 101723, doi: 10.1016/j.aei.2022.101723.
7. B. Mao, F. Tang, Y. Kawamoto, and N. Kato, "AI models for green communications towards 6G," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 1, pp. 210–247, Firstquarter 2022, doi: 10.1109/COMST.2021.3130901.
8. P. P. Ray, "A review on TinyML: State-of-the-art and prospects," *J. King Saud. Univ. Comput. Inf. Sci.*, vol. 34, no. 4, pp. 1595–1623, 2022, doi: 10.1016/j.jksuci.2021.11.019.
9. N. Schizas, A. Karras, C. Karras, and S. Sioutas, "TinyML for ultra-low power AI and large scale IoT deployments: A systematic review," *Future Internet*, vol. 14, no. 12, Art. no. 363, 2022, doi: 10.3390/fi14120363.
10. Z. Wang, P. Reviriego, F. Niknia, J. Conde, S. Liu, and F. Lombardi, "Adaptive resolution inference (ARI): Energy-efficient machine learning for internet of things," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14,076–14,087, Apr. 2024, doi: 10.1109/JIOT.2023.3339623.
11. E. Baram et al. "tinyML deployment working group white paper #1." tinyML Foundation. Accessed: Feb. 1, 2024. [Online]. Available: https://www.tinyml.org/static/98111ec2e44e63079e10872b485777a0/tinyML_Deployment_WG_White_Paper_1.pdf
12. A. B. Kolltveit and J. Li, "Operationalizing machine learning models: A systematic literature review," in *Proc. EEE/ACM 1st Int. Workshop Softw. Eng. Responsible Artif. Intell. (SE4RAI)*, Pittsburgh, PA, USA. New York, NY, USA: Association for Computing Machinery, 2023, pp. 1–8.
13. A. Barrak, F. Petrillo, and F. Jaafar, "Serverless on machine learning: A systematic mapping study," *IEEE Access*, vol. 10, pp. 99,337–99,352, 2022, doi: 10.1109/ACCESS.2022.3206366.
14. M. Bauer, "FIWARE: Standard-based open source components for cross-domain IoT platforms," in *Proc. IEEE 8th World Forum Internet Things (WF-IoT)*, 2022, pp. 1–6, doi: 10.1109/WF-IoT54382.2022.10152259.
15. V. Janapa Reddi et al., "Edge impulse: An MLOps platform for tiny machine learning," in *Proc. Mach. Learn. Syst.*, vol. 5, 2023, pp. 1–15.

JAVIER CONDE is an assistant professor with Departamento de Ingenieria de Sistemas Telematicos, Escuela Tecnica Superior de Ingenieros de Telecomunicacion, Universidad Politécnica de Madrid, 28040, Madrid, Spain. His research interests include digital twins and machine learning. Conde received his Ph.D. degree in telematics engineering from Universidad Politécnica de Madrid. Contact him at javier.conde.diaz@upm.es.

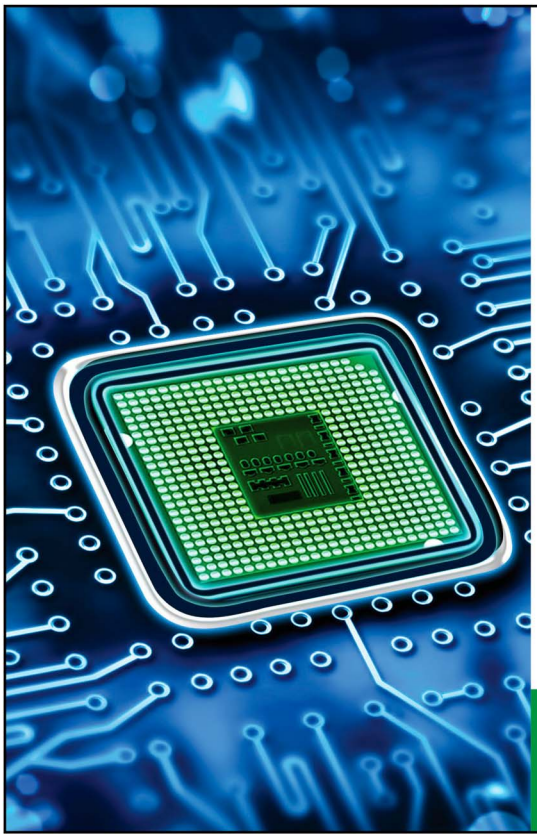
ANDRÉS MUNOZ-ARCENTALES is an assistant professor with Departamento de Ingenieria de Sistemas Telematicos, Escuela Tecnica Superior de Ingenieros de Telecomunicacion, Universidad Politécnica de Madrid, 28040, Madrid, Spain. His research interests include data engineering and data usage control. Munoz received his Ph.D. degree in telematics engineering from Universidad Politécnica de Madrid. Contact him at joseandres.munoz@upm.es.

ÁLVARO ALONSO is an associate professor with Departamento de Ingenieria de Sistemas Telematicos, Escuela Tecnica Superior de Ingenieros de Telecomunicacion, Universidad Politécnica de Madrid, 28040, Madrid, Spain. His research

interests include multivideoconferencing systems and security management. Alonso received his Ph.D. degree in telecommunications engineering from Universidad Politécnica de Madrid. Contact him at alvaro.alonso@upm.es.

JOAQUÍN SALVACHÚA is an associate professor with Departamento de Ingenieria de Sistemas Telematicos, Escuela Tecnica Superior de Ingenieros de Telecomunicacion, Universidad Politécnica de Madrid, 28040, Madrid, Spain. His research interests include cloud and edge architectures. Salvachúa received his Ph.D. degree in telecommunications engineering from Universidad Politécnica de Madrid. Contact him at joaquin.salvachua@upm.es.

GABRIEL HUECAS is an associate professor with Departamento de Ingenieria de Sistemas Telematicos, Escuela Tecnica Superior de Ingenieros de Telecomunicacion, Universidad Politécnica de Madrid, 28040, Madrid, Spain. His research interests include digitization and data spaces. Huecas received his Ph.D. degree in telecommunications engineering from Universidad Politécnica de Madrid. Contact him at gabriel.huecas@upm.es.



IEEE TRANSACTIONS ON

COMPUTERS

Call for Papers:
IEEE Transactions on Computers

Publish your work in the IEEE Computer Society's flagship journal, *IEEE Transactions on Computers*. The journal seeks papers on everything from computer architecture and software systems to machine learning and quantum computing.

Learn about calls for papers and submission details at
www.computer.org/tc.

IEEE COMPUTER SOCIETY

IEEE